INSTITUTIONAL ENGINEERING BLUEPRINT

Multi-Asset, Multi-Strategy Quantitative Research to Deployment and Monitoring Monorepo

Implementation roadmap for merge requests, quality gates, production controls, and institutional readiness

Scope. Research data and experiments; point-in-time datasets; feature and model lifecycle; deterministic simulation; portfolio construction and risk; OMS/EMS and venue adapters; immutable deployment; observability; security; model governance; incident response; and 96 sequenced implementation merge requests.

Positioning. This is an independent target architecture informed by public engineering principles from leading quantitative trading firms and by broader software, SRE, model-risk, and algorithmic-trading standards. It does not describe or claim access to any firm's proprietary internal systems.

Version 1.0
25 August 2026

Document status and use

This document is a build plan, not a product brochure. It makes normative recommendations using the terms **MUST**, **SHOULD**, and **MAY**. A mandatory gate may only be waived through the documented exception process; a score or dashboard does not compensate for a failed hard control.

The reference architecture is intentionally technology-opinionated enough to be implementable, but the contracts and controls are more important than the named tools. A reasonable reference stack is GitLab merge requests and protected branches; Bazel for hermetic polyglot builds; Nix or equivalent for developer environments; Python for research and orchestration; C++ or Rust for latency-sensitive components; Protobuf and Apache Arrow/Parquet for contracts and analytical data; an Iceberg-style point-in-time lakehouse; Kafka/Redpanda-class event transport; MLflow-class registry; OpenLineage-class lineage; Kubernetes/GitOps for standard services; an immutable bare-metal lane for ultra-low latency; and OpenTelemetry with Prometheus/Grafana-class observability. Substitutions are valid only when they preserve the invariants in this guide.

The regulatory mapping is illustrative and must be tailored by the relevant legal, compliance, market-structure, tax, and operational-risk functions. Public materials from Jane Street, Hudson River Trading, and Citadel Securities support general themes such as rigorous review, reproducible deployment, close research-engineering integration, distributed high-performance systems, and production monitoring. Their detailed internal architectures are proprietary and are not represented here.

Contents

Document status and use i

1 Executive blueprint 1

1.1 What institutional grade means . . . 1
1.2 The seven non-negotiable invariants . . . 1
1.3 Recommended program shape . . . 1
1.4 First production target . . . 2

2 Scope, assumptions, and operating model 3

2.1 Supported problem classes . . . 3
2.2 Explicit non-goals . . . 3
2.3 Service and change criticality . . . 3
2.4 MR risk classes . . . 3
2.5 Accountabilities . . . 4

3 Target architecture and monorepo topology 5

3.1 Control plane, data plane, and runtime lanes . . . 5
3.2 The thin waist . . . 6
3.3 Permitted dependency direction . . . 6
3.4 Reference repository layout . . . 6
3.5 Build and reproducibility architecture . . . 7

4 Canonical multi-asset domain and point-in-time data 8

4.1 Instrument identity is temporal . . . 8
4.2 Time semantics and causal correctness . . . 8
4.3 Data states and publication policy . . . 8
4.4 Asset-specific contract matrix . . . 9

5 Research, validation, and model governance 10

5.1 The research contract . . . 10
5.2 Committed trial ledger . . . 10
5.3 Validation sequence . . . 10
5.4 Train, validation, test, and final OOS . . . 10
5.5 Objective and economic accounting . . . 11
5.6 Model card and independent validation . . . 11

6 Simulation, portfolio construction, risk, and execution 12

6.1 Deterministic event model . . . 12
6.2 Reality gap management . . . 12
6.3 Accounting and risk invariants . . . 12
6.4 Independent pre-trade risk . . . 12

6.5 Order-state resilience . . . 13

7 Merge-request governance and quality gates 14

7.1 Review philosophy . . . 14
7.2 Definition of Ready . . . 14
7.3 Required MR size and shape . . . 14
7.4 Quality gate catalog . . . 15
7.5 Mandatory gates by risk class . . . 16
7.6 Waiver policy . . . 16
7.7 Definition of Done . . . 16

8 Deployment and production activation 17

8.1 Immutable release manifest . . . 17
8.2 Promotion state machine . . . 17
8.3 Standard-service and low-latency lanes . . . 17
8.4 Progressive rollout and rollback . . . 18
8.5 Disaster recovery . . . 18

9 Monitoring, SLOs, and incident control 19

9.1 Production control loop . . . 19
9.2 Monitoring planes . . . 19
9.3 SLO design . . . 20
9.4 PnL and TCA attribution . . . 20
9.5 Alert design . . . 20
9.6 Incident lifecycle . . . 20

10 Security, model risk, and regulatory alignment 21

10.1 Secure software supply chain . . . 21
10.2 Access and separation of duties . . . 21
10.3 Algorithmic trading controls . . . 21
10.4 Model-risk operating principles . . . 21
10.5 Control evidence map . . . 22

11 Implementation roadmap: 96 merge requests 23

11.1 How to use the roadmap . . . 23
11.2 Illustrative milestones . . . 23
11.3 Detailed MR sequence . . . 24
11.4 Wave exit reviews . . . 33
11.5 Resourcing and sequencing . . . 33

12 Asset and strategy certification playbooks 34

12.1 Asset onboarding checklist . . . 34
12.2 Strategy manifest . . . 34
12.3 Strategy-family certification matrix . . . 35
12.4 Capital scaling protocol . . . 36

13 Implementation templates **37**
13.1 Merge request template . . . 37
13.2 CODEOWNERS example . . . 37
13.3 Conceptual CI pipeline . . . 37
13.4 Dataset contract example . . . 38
13.5 Strategy manifest example . . . 38
13.6 Release manifest example . . . 39

14 Institutional readiness scorecard **40**
14.1 Maturity levels . . . 40
14.2 Final launch checklist . . . 40

A Failure-mode control matrix **41**

B Selected references **42**

Chapter 1

Executive blueprint

1.1 What institutional grade means

An institutional quantitative platform is not defined by a language, database, model class, or cloud provider. It is institutional grade when the firm can prove, for each production decision and order, that the system was:

1. **Causally correct**: only information available at decision time could affect the decision.
2. **Reproducible**: source, build, data, model, configuration, and random state can reproduce the result within declared tolerance.
3. **Independently controlled**: capital-at-risk changes require review and risk controls outside the strategy process.
4. **Implementable**: backtests include executable timing, liquidity, borrow, fees, impact, capacity, and venue behavior.
5. **Observable**: data, model, execution, risk, and economic health are correlated to the exact release.
6. **Recoverable**: state, orders, positions, and configuration survive faults without duplication or silent loss.
7. **Auditable**: evidence links hypothesis, trials, code, data, validation, approvals, deployment, orders, incidents, and retirement.

The central architectural thesis

Use one source of truth for code, contracts, lineage, ownership, and change governance, but maintain multiple deployables, security zones, performance profiles, and failure domains. **Monorepo does not mean monolith.**

1.2 The seven non-negotiable invariants

No.	Invariant	Operational consequence
1	Point-in-time causality	Every feature, label, universe, price, corporate action, and reference-data lookup carries knowledge-time and can be queried as known at the decision timestamp.
2	One strategy kernel	Research, replay, paper, shadow, canary, and live use the same decision logic; differences are injected adapters, not copied implementations.
3	Immutable release identity	A release is a signed manifest of code, build, model, data references, schema, config, limits, evidence, and approvals. Production never deploys a mutable tag or workstation artifact.
4	Risk outside the strategy	Independent risk gates, limits, throttles, and kill switches can stop or constrain the strategy even if the strategy process is faulty.
5	Deterministic replay	Market, reference, order, fill, state, and control events can be replayed with specified ordering, clocks, and random streams.
6	Evidence before promotion	Promotion requires complete test and review evidence; a profitable backtest is neither necessary nor sufficient for operational approval.
7	Monitor the economic chain	Platform health, data quality, model behavior, execution quality, risk, and PnL attribution are monitored together and keyed by release.

1.3 Recommended program shape

The program should be executed as twelve waves and 96 logical merge requests. Governance, build, contracts, security, and observability begin early; they are not a cleanup phase. The critical path is canonical contracts $\rightarrow$ point-in-time data $\rightarrow$ deterministic simulation $\rightarrow$ portfolio/risk $\rightarrow$ trading runtime $\rightarrow$ controlled deployment $\rightarrow$ asset and strategy certification.

An illustrative implementation with 12–16 core engineers, 5–8 quantitative researchers, and dedicated part-time or full-time risk, SRE, security, and validation support is typically a 12–18 month program. A smaller team should reduce initial asset classes and latency profiles rather than weaken controls. Calendar estimates are planning assumptions, not guarantees.

1.4 First production target

The first live target should be deliberately narrow: one asset class, one strategy family, one or two venues/brokers, bounded capital, and a non-ultra-low-latency path unless latency is the strategy's primary edge. The objective is to certify the platform's control loop, not to maximize initial strategy breadth.

Chapter 2

Scope, assumptions, and operating model

2.1 Supported problem classes

The architecture supports multi-asset strategies spanning cross-sectional alpha, time-series forecasting, statistical arbitrage, relative value, market making, execution algorithms, volatility and options, macro and event-driven models, and machine-learning-based forecasts. The design distinguishes forecast horizon and execution horizon; a daily cross-sectional model and a microsecond market maker may share contracts, registries, data lineage, and governance while using different runtimes and performance budgets.

2.2 Explicit non-goals

- One universal runtime for all latencies and asset classes.
- A single database that serves research, order routing, audit, and monitoring equally well.
- Automatic promotion based only on Sharpe ratio or PnL.
- Manual production deployment, notebook execution, or package installation.
- Treating monitoring as a dashboard project detached from limits, rollback, or incident ownership.
- Claiming that process quality guarantees investment performance.

2.3 Service and change criticality

Tier	Examples	Failure consequence	Minimum posture
T0	Pre-trade risk, OMS, kill switches, venue sessions, authoritative positions	Uncontrolled orders, unbounded exposure, inability to stop trading	Independent control path, fail-closed defaults, deterministic recovery, active on-call, game days, strict change class.
T1	Live strategy runtime, market data, feature serving, model service, deployment control	Incorrect or stale decisions, missed trading, distorted risk	Tight SLOs, shadow/canary, state recovery, release identity, data/model monitoring.
T2	Batch research, simulation clusters, catalog, experiment services	Research delay, lost productivity, delayed validation	Reproducible jobs, quotas, restore, lineage, monitored quality; less stringent availability than T0/T1.
T3	Documentation, examples, offline utilities	Local productivity or knowledge loss	Version control, ownership, tests appropriate to impact.

2.4 MR risk classes

Class	Typical change	Required review	Required rollout
R0	Documentation or non-functional metadata	One owner; automated checks	Merge after fast lane.
R1	Research-only code with no shared data or production dependency	Research Code Owner plus peer	Reproducible batch test; no production promotion.
R2	Shared library, schema, dataset, feature, or model candidate	Code Owner plus independent domain reviewer; data/model reviewer when material	Merged-result pipeline; compatibility, replay, and impact evidence.

R3	Paper/shadow runtime, deployment tooling, non-capital production service	Code Owner, SRE/platform, security as applicable	Staging, failure injection, shadow or synthetic validation.
R4	Capital-at-risk strategy, OMS, risk, venue, limits, prod access, kill switch	Two-person rule with independent risk/validation and security/platform where applicable	Full certification, paper/shadow, bounded canary, signed activation, rollback rehearsal.

2.5 Accountabilities

The platform should organize around persistent ownership, not a central ticket queue. A practical model uses core platform, data/reference, research/simulation, portfolio/risk, trading connectivity, SRE/security, and asset/strategy pods. Quantitative researchers own economic hypotheses and research specifications; engineers own durable interfaces and operational behavior; risk owns independent limits and challenge; model validation owns effective challenge for material models; SRE owns reliability mechanisms; security owns trust and access controls; traders or portfolio managers own commercial use within the signed envelope.

No hand-off gap

The researcher who proposes a model and the engineer who productionizes it remain jointly accountable through shadow and early live operation. Production incidents and realized transaction-cost gaps feed back into the research ledger and simulator.

Chapter 3

Target architecture and monorepo topology

3.1 Control plane, data plane, and runtime lanes

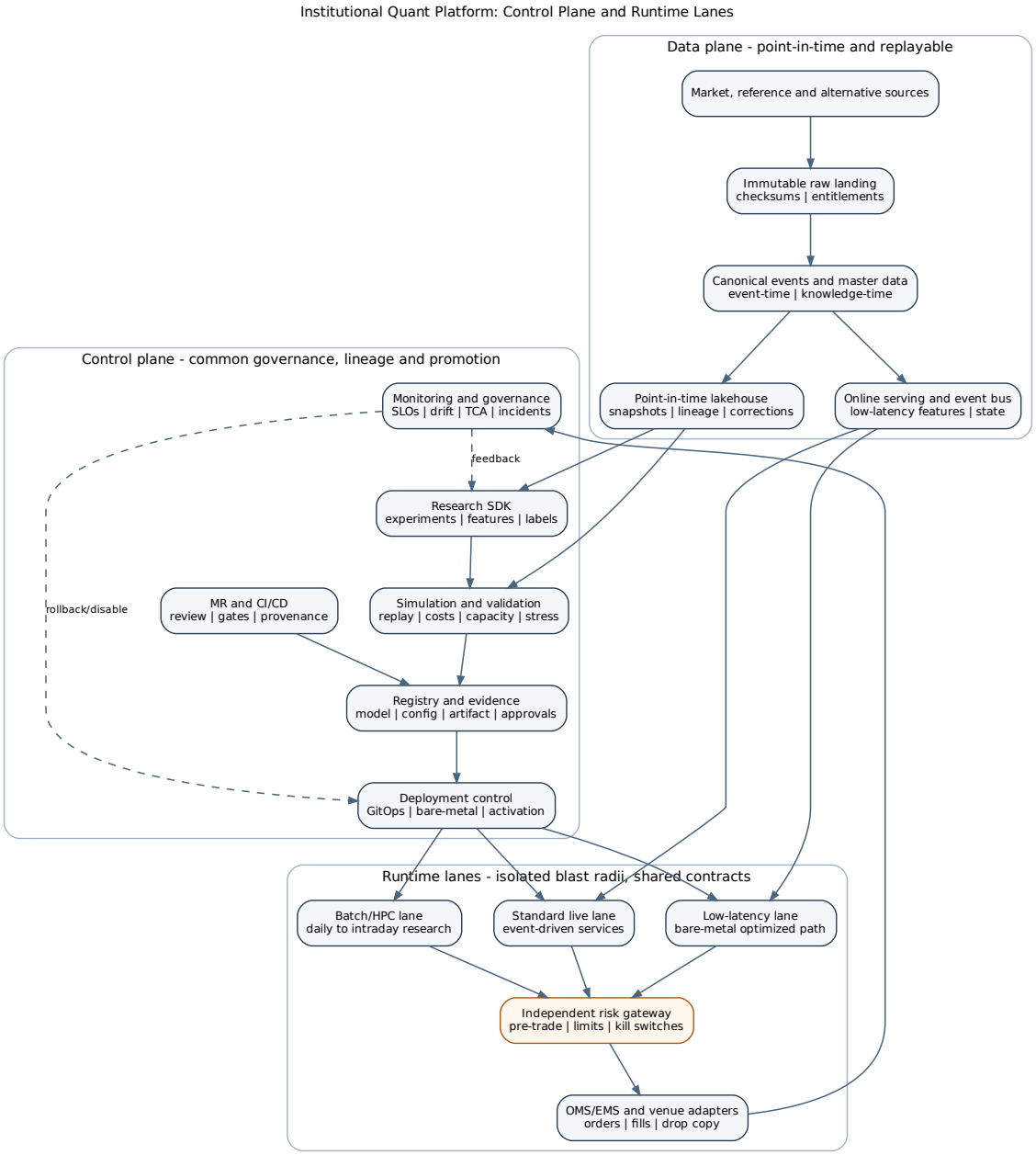


Figure 3.1: Unified governance with isolated runtime lanes.

The control plane manages experiments, evidence, artifacts, promotion, configuration, and monitoring. The data plane preserves raw inputs, canonical events, point-in-time datasets, and low-latency serving. Runtime lanes are separated by latency and operational needs. The ultra-low-latency path should not carry general-purpose control-plane dependencies

in the hot path, but it must still report the same release identity, consume compatible schemas, respect external risk, and emit audit/telemetry.

3.2 The thin waist

The platform's stable integration surface consists of:

1. Canonical identifiers, units, timestamps, and event contracts.
2. A strategy kernel interface separating features, forecasts, targets, and execution intent.
3. An independent risk decision contract.
4. A release manifest and evidence schema.
5. Correlated telemetry and audit event schemas.

Everything else may evolve behind adapters. This prevents asset-specific details, infrastructure choices, and venue protocols from contaminating the entire repository.

3.3 Permitted dependency direction

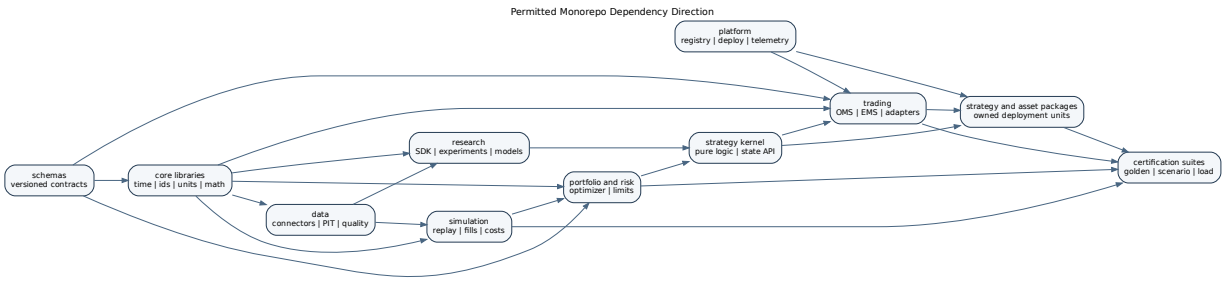


Figure 3.2: Acyclic dependency direction enforced by the build graph.

Key restrictions are normative:

- `schemas` and foundational libraries never depend on research, strategy, trading, or deployment packages.
- Production strategy packages never import notebooks, ad hoc analysis folders, or workstation configuration.
- Risk checks and kill switches do not depend on the strategy process being healthy.
- Venue-specific code is isolated behind certified adapters.
- Asset-specific accounting and lifecycle behavior implement canonical interfaces and carry golden fixtures.
- Environment-specific configuration is data, not compiled conditional logic.

3.4 Reference repository layout

```
/
  MODULE.bazel                # pinned polyglot build graph
  .gitlab-ci.yml              # affected-target and certification pipelines
  CODEOWNERS
  docs/
    architecture/             # principles, ADRs, domain maps
    runbooks/                 # generated and reviewed operational procedures
    controls/                 # control objectives and evidence mappings
  schemas/
    identity/ market/ reference/ order/ fill/ position/ risk/ audit/
  core/
    time/ ids/ units/ numerics/ serialization/ randomness/ telemetry/
  data/
    contracts/ connectors/ normalization/ quality/ point_in_time/
    lineage/ catalog/ replay/ corrections/
```

```
research/
  sdk/ datasets/ features/ labels/ experiments/ models/ reports/
simulation/
  engine/ market_models/ fills/ costs/ scenarios/ accounting/
portfolio/
  forecasts/ optimizers/ constraints/ allocation/
risk/
  models/ pretrade/ limits/ stress/ liquidity/ reconciliation/
trading/
  strategy_kernel/ runtime/ oms/ ems/ fix/ venue_adapters/ drop_copy/
platform/
  registry/ promotion/ config/ deploy/ observability/ secrets/
assets/
  equities/ futures/ options/ fx/ rates/ credit/ digital_assets/
strategies/
  cross_sectional/ stat_arb/ market_making/ execution/ volatility/
tests/
  unit/ property/ golden/ replay/ scenario/ integration/ load/ chaos/
environments/
  dev/ ci/ replay/ staging/ paper/ shadow/ canary/ production/
tools/
  cli/ codegen/ migrations/ evidence/ release/
```

3.5 Build and reproducibility architecture

A large monorepo only works with strong tooling. Public Google research describes a monolithic repository as a common source of truth while emphasizing the supporting systems and trade-offs required at scale [5]. Bazel's hermeticity model relies on explicit inputs and action sandboxing so local and remote builds behave consistently [6]. Nix similarly emphasizes isolated, reproducible, declarative environments and rollback [7].

The build system should produce a content-addressed artifact from explicit source, toolchain, dependency, and generation inputs. It should support affected-target testing for speed, full merged-result testing for integration safety, remote execution for scale, and a complete provenance attestation for promotion.

Chapter 4

Canonical multi-asset domain and point-in-time data

4.1 Instrument identity is temporal

An internal stable identifier must not be a ticker. The domain should distinguish economic instrument, listing, contract, venue symbol, share class, derivative series, and vendor identifier. Mappings are validity-dated and source-attributed. Symbol reuse, corporate actions, contract rolls, relistings, and vendor corrections must be represented rather than overwritten.

4.2 Time semantics and causal correctness

For a decision made at time t_d, every input x must satisfy

$$t_{\text{knowledge}}(x) \leq t_d. \tag{4.1}$$

Event time describes when the economic or market event occurred; knowledge time describes when the system could have known it; ingestion time describes arrival into a platform; valid time describes the business period to which reference data applies. These timestamps are not interchangeable.

A robust record contains at least:

- source event time and precision;
- source sequence or file revision;
- platform receipt and ingestion times;
- knowledge time used for causal queries;
- valid-from and valid-to for reference data;
- source, transformation version, checksum, and correction lineage;
- clock quality and timezone/convention metadata.

Lookahead often enters through revised fundamentals, backfilled symbology, current index constituents, final corporate-action terms, end-of-day snapshots used before availability, global scalers fit on all dates, labels overlapping validation windows, survivorship-biased universes, and transaction-cost estimates calibrated with future liquidity. Each requires an automated adversarial test, not a reviewer assumption.

4.3 Data states and publication policy

State	Purpose	Required properties
Raw immutable	Legal and technical source of record	Exact payload and checksum; source/license and arrival metadata; encryption, retention, and append-only storage.
Canonical normalized	Cross-asset semantic interoperability	Versioned schema and source pointer; deterministic transform; units, time semantics, and quality status.
Curated point-in-time	Research, backtest, and batch production	As-of joins; universe membership; revision policy; snapshot digest; lineage and reproducibility.
Online serving	Low-latency production decisions	Bounded freshness; feature parity; explicit state and fallback; version identity and telemetry.
Correction stream	Controlled restatement and impact analysis	Old/new value, reason, and knowledge time; impacted datasets/models/releases; idempotent backfill.

4.4 Asset-specific contract matrix

Asset	Identity/lifecycle	Market data	Trading/accounting	Mandatory simulation and monitoring
Cash equities	Listings, share classes, corporate actions, delist/relist, borrow/locate, settlement	Trades, quotes, books, auctions, halts, tick/lot and limit rules	Partial fills, fees/rebates, short sale controls, borrow/funding, dividends, FX	Auction and halt scenarios; corporate-action replay; borrow shocks; spread/impact and reconciliation.
Futures	Contract specs, multiplier, expiry, first notice, delivery, spread, continuous mapping	Session breaks, limit states, book, implieds where available	Variation margin, roll, settlement, exchange fees, calendar spreads	Roll leakage tests; expiry/default scenarios; margin and gap stress; session recovery.
Listed options	Series, strike, expiry, style, deliverable, adjustment, exercise/assignment	Chains, books, implied volatility, dividends/rates/borrow inputs	Greeks, premium, exercise, assignment, pin risk, complex orders, margin	Surface consistency; nonlinear stress; expiry/assignment; hedge and volatility PnL attribution.
FX	Pair, tenor, spot date, forward points, NDF fixing, value-date calendars	Fragmented quotes, swaps/forwards, fixing sources	Notional currencies, settlement, rollover, counterparty/credit, broken dates	Calendar/convention fixtures; fixing and settlement scenarios; venue/counterparty exposure.
Rates/credit	Curves, instruments, coupons, day count, calls, reference rates, settlement	Quotes, evaluated prices, curves, fixings, liquidity proxies	Accrual, clean/dirty price, cash flows, collateral, counterparty, lifecycle	Curve-shock and spread stress; cash-flow reconciliation; stale/evaluated-price controls.
Digital assets	Token/chain/venue identity, forks, redenomination, custody state	24/7 books/trades, index/funding, on-chain or venue events	Exchange/custody credit, funding, liquidations, transfer/custody, stablecoin FX	Venue/chain outage, liquidation cascade, custody and counterparty limits, continuous on-call.

Chapter 5

Research, validation, and model governance

5.1 The research contract

Every strategy proposal should state, before model search:

- economic mechanism and falsifiable hypothesis;
- prediction target, sign convention, horizon, and label availability;
- decision and execution horizons;
- asset universe, eligibility, and rebalance policy;
- benchmark and minimum viable baseline;
- portfolio bet structure and risk neutralization;
- expected turnover, liquidity, borrow, financing, and capacity constraints;
- evaluation metrics and rejection criteria;
- intended production mode, monitoring, and kill conditions.

5.2 Committed trial ledger

The experiment ledger is a control against hidden selection. It records all material attempts, including failed feature sets, alternative universes, hyperparameters, transformations, and cost assumptions. The effective number and dependence of trials inform selection-bias adjustments. Bailey et al. formalize the probability of backtest overfitting, while the Deflated Sharpe Ratio adjusts for selection bias, non-normality, and sample length [12, 13]. Harvey, Liu, and Zhu show why conventional significance thresholds become unreliable after extensive factor search [14].

5.3 Validation sequence

1. **Causal integrity**: point-in-time data, universe, label, fit/transform, and overlap tests.
2. **Statistical validity**: dependence-aware uncertainty, nested selection, purging/embargo, trial adjustment, calibration, and residual diagnostics.
3. **Economic validity**: mechanism, monotonicity where expected, ablations, regime behavior, and plausible source of compensation.
4. **Implementability**: timing, fees, spread, queue/fill, impact, borrow, funding, market impact, and capacity.
5. **Portfolio/risk validity**: exposures, concentration, covariance instability, liquidity, margin, stress, and interaction with existing books.
6. **Operational validity**: deterministic replay, state recovery, data degradation, venue failure, monitoring, runbook, and rollback.

5.4 Train, validation, test, and final OOS

Use nested time-aware selection. Training fits parameters and transformations. Validation selects specifications. A test period estimates performance after selection. A final untouched OOS period is used once for a promotion decision; repeated inspection converts it into validation data and must be recorded as another trial. Overlapping labels require purging and an embargo appropriate to the information horizon.

5.5 Objective and economic accounting

A production candidate should be selected on a pre-specified, cost-aware objective, not gross Sharpe alone. One generic form is

$$\widehat{J}(\theta) = \widehat{\mathbb{E}}[\Pi_\theta] - \lambda_r \widehat{\mathcal{R}}_\theta - \lambda_t \widehat{\mathrm{TC}}_\theta - \lambda_u \widehat{\mathcal{U}}_\theta, \tag{5.1}$$

where $\mathcal{R}$ is a risk functional, TC is transaction and financing cost, and $\mathcal{U}$ penalizes instability, complexity, or operational burden. The weights and metrics are declared before final OOS.

Total execution and holding cost should be decomposed as

$$C = C_{\text{spread}} + C_{\text{fees}} + C_{\text{latency}} + C_{\text{impact}} + C_{\text{timing}} + C_{\text{funding}} + C_{\text{borrow}} + C_{\text{hedge}}. \tag{5.2}$$

Cost parameters are segmented by asset, venue, order type, urgency, volatility, participation, and liquidity regime. Novy-Marx and Velikov demonstrate that transaction costs materially change anomaly profitability and data-snooping concerns [15].

5.6 Model card and independent validation

The promotion card contains intended use, prohibited use, model and feature versions, data and label lineage, limitations, sensitivity, validation results, benchmark deltas, capacity, risk, monitoring, fallback, and owner. The 2026 U.S. interagency model-risk guidance emphasizes a risk-based approach, clear purpose, testing including out-of-sample or out-of-time analysis where appropriate, validation, documentation, governance, and ongoing monitoring [16]. The platform should apply these principles proportionately even where the specific guidance is not legally applicable.

A candidate passes only if every mandatory gate is true. A weighted score is useful for prioritization, but the release condition is logically

$$\text{Promotable} = \prod_{i \in \mathcal{M}} g_i = 1, \tag{5.3}$$

where $g_i \in \{0, 1\}$ are mandatory gates. Expected PnL cannot offset a failed causal, risk, security, or recovery gate.

Chapter 6

Simulation, portfolio construction, risk, and execution

6.1 Deterministic event model

The simulator should use the same canonical events and strategy kernel as live operation. It must distinguish source event, system receipt, decision, order send, venue acknowledgement, and fill times. Event-ordering rules are documented for simultaneous timestamps. Randomness is partitioned into named streams so adding a diagnostic does not change simulated fills.

6.2 Reality gap management

No simulator fully reproduces market interaction. The correct control is to measure and manage the reality gap:

- calibrate fill, queue, latency, reject, and impact models to live or high-quality reference data;
- compare replay, paper, shadow, canary, and live decision traces;
- maintain conservative stress bands around uncertain parameters;
- track simulated versus realized TCA by release and strategy;
- feed material gaps back into research conclusions and capacity limits.

6.3 Accounting and risk invariants

Examples of property-based invariants include:

$$V_t = C_t + \sum_i q_{i,t} P_{i,t} + A_t, \tag{6.1}$$

$$q_{i,t} = q_{i,t-1} + \sum_{f \in \mathcal{F}_{i,t}} s_f |f|, \tag{6.2}$$

$$\text{remaining_qty} \geq 0, \tag{6.3}$$

$$\text{terminal order} \Rightarrow \text{no further executable transition}, \tag{6.4}$$

$$\text{duplicate event id} \Rightarrow \Delta\text{state} = 0. \tag{6.5}$$

Here A_t includes accruals, fees, funding, and corporate-action adjustments under explicit sign conventions.

Portfolio constraints may include

$$w^\top \Sigma w \leq \sigma_{\max}^2, \qquad |Aw| \leq b, \qquad \ell^\top |w| \leq L, \tag{6.6}$$

with liquidity, borrow, margin, turnover, concentration, and asset-specific constraints. Solver diagnostics and infeasibility handling are first-class evidence.

6.4 Independent pre-trade risk

Risk checks execute outside the strategy process and consume authoritative positions plus pending orders. Timeouts, stale inputs, or unavailable limits default to a documented safe state, generally fail-closed for new risk. Hard controls include maximum order size/notional, price collars, position, leverage, concentration, factor/greek exposure, credit, message rate, borrow/locate, venue/account authorization, and kill status.

6.5 Order-state resilience

The OMS state machine must handle duplicate, delayed, missing, and out-of-order messages; partial fills; amend/cancel races; session reset; venue disconnect; broker bust/correct; drop-copy divergence; and restart. Idempotency keys, sequence tracking, state fencing, and independent reconciliation prevent double orders and phantom positions.

Chapter 7

Merge-request governance and quality gates

7.1 Review philosophy

Jane Street's public discussion of code review describes rigorous per-file scrutiny, ownership, a monorepo context, and preserved deployed versions for rollback [1]. Its public XAR discussion emphasizes reproducible self-contained Python environments and deployment artifacts [2]. Public HRT and Citadel Securities materials similarly emphasize close collaboration among research and engineering, high-performance distributed systems, simulation, risk, and robust monitoring [3, 4]. These are principles, not claims about exact internal implementations.

GitLab supports required approvals, Code Owners on protected branches, merged-result pipelines, and merge trains that test integrated changes before merge [8, 9, 10, 11]. The platform should configure equivalent controls even when another forge is used.

7.2 Definition of Ready

An MR is ready for implementation only when it has:

- linked issue or roadmap item, owner, risk class, and impacted systems;
- explicit scope and non-goals;
- approved ADR for cross-cutting change;
- data, model, schema, security, and regulatory impact analysis;
- test and benchmark plan with acceptance thresholds;
- migration, rollout, rollback, monitoring, and runbook plan;
- dependency and reviewer availability.

7.3 Required MR size and shape

Prefer one logical behavior change with tests and evidence. As a review target, keep non-generated changes near or below 400 lines where practical; larger changes require a split plan, generated-code explanation, or an explicitly approved indivisible migration. MRs must not combine broad refactoring, schema migration, model change, and production activation unless the sequencing itself is the subject of the MR.



Figure 7.1: A capital-at-risk MR accumulates machine-verifiable evidence before promotion.

7.4 Quality gate catalog

Gate	Control objective	Representative checks	Blocking evidence
QG01	Source hygiene	Format, lint, type checks, warnings-as-errors, forbidden APIs, generated-code separation	Clean reports; suppression has owner and expiry.
QG02	Hermeticity/reproducibility	Pinned toolchains/dependencies, sandbox, deterministic generation, no undeclared input	Rebuild digest comparison; provenance.
QG03	Functional correctness	Unit, contract, state-machine, serialization, boundary and error-path tests	Required tests pass on affected and merged result.
QG04	Property/numerical correctness	Property/metamorphic tests, NaN/Inf, tolerance policy, conditioning, gradient/perturbation	Invariants and numeric report within budget.
QG05	Schema/data/PIT integrity	Compatibility, freshness, completeness, uniqueness, causal as-of, universe, revision, lineage	Dataset fingerprint; leakage suite; compatibility report.
QG06	Research/model validity	Pre-registered metrics, baselines, nested time split, trial ledger, DSR/PBO/FDR, ablations, stability	Independent research/validation report.
QG07	Costs/capacity	Spread, fees, impact, latency, borrow, funding, liquidity, participation and scale stress	Calibrated cost/capacity report with sensitivity.
QG08	Portfolio/risk controls	Exposures, limits, stress, margin, concentration, liquidity, fail-safe behavior	Risk approval; limit and scenario test evidence.
QG09	Integration/scenario	Replay, OMS/venue, disconnect, duplicate, gap, bust/correct, reconcile, paper/shadow	Scenario matrix and trace parity.
QG10	Performance	Tail latency, throughput, jitter, memory, CPU, allocations, startup/recovery, query performance	Benchmark against signed budget; no unexplained regression.
QG11	Resilience	Crash/restart, backpressure, failover, degraded mode, backup/restore, chaos, clock/data failure	Recovery evidence meets RTO/RPO and state invariants.
QG12	Security/supply chain	SAST, secrets, dependency/vulnerability/license, SBOM, signature, provenance, IAM, network	No unaccepted critical finding; verified artifact policy.
QG13	Documentation/governance	ADR, API docs, examples, model card, runbook, control mapping, ownership	Complete links; docs/examples compile.
QG14	Observability/SLO	Metrics, logs, traces, audit, dashboards, alerts, cardinality, SLO and owner	Synthetic alert and dashboard evidence.
QG15	Deployment/promotion	Staging, merged result, paper, shadow, canary, rollout, rollback, activation envelope	Signed release manifest and post-deploy validation.

7.5 Mandatory gates by risk class

Class	Mandatory gates	Minimum approvers
R0	QG01, QG13	One Code Owner.
R1	QG01–QG06, QG13; QG10 where computationally material	Research owner plus peer.
R2	QG01–QG06, QG09–QG14; QG07–QG08 when economic/risk relevant	Code Owner plus independent data/model/domain reviewer.
R3	QG01–QG05, QG09–QG15; QG06–QG08 when model/trading relevant	Code Owner, platform/SRE, security as applicable.
R4	QG01–QG15	Domain owner, independent risk/model validation, platform/SRE, and security/compliance where applicable; dual control for activation.

7.6 Waiver policy

A waiver contains a unique ID, failed gate, rationale, affected release and scope, compensating controls, owner, approvers, expiry, and remediation MR. It is visible in the release manifest and monitoring. No indefinite waiver is allowed. Causal integrity, artifact identity, independent hard risk, and kill capability are not waivable for live capital.

7.7 Definition of Done

An MR is done only after code and tests merge, documentation and runbooks are updated, evidence is stored, release identity is available, monitoring is active, post-deploy checks pass, and temporary flags or migrations have an owner and removal date. A merged MR is not automatically a production release.

Chapter 8

Deployment and production activation

8.1 Immutable release manifest

A release manifest should include:

- repository commit and reviewed MR identifiers;
- build platform, toolchain, dependency lock, provenance, SBOM, signature, and artifact digest;
- model, feature graph, dataset snapshot references, schema versions, random-state policy, and validation report;
- configuration and limit digests; target environment, accounts, venues, assets, universe, capital, and schedule;
- required service versions and compatibility range;
- dashboards, alerts, runbooks, rollback target, kill procedure, and approvals.

Deployment and activation are separate. A binary may be deployed but unable to trade until an independently authorized activation envelope is applied. The process reports its release identity continuously and includes it in every decision, order, fill, risk, and audit event.

8.2 Promotion state machine

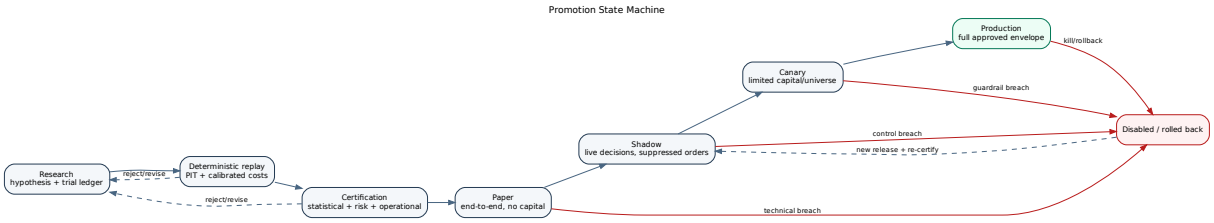


Figure 8.1: Capital exposure rises only after progressively stronger evidence.

8.3 Standard-service and low-latency lanes

Dimension	Standard service lane	Low-latency lane
Typical runtime	Kubernetes or equivalent orchestrator	Bare metal or tightly controlled host runtime
Promotion	GitOps, signed image/manifests, policy admission	Signed package/host profile, staged host groups, same release registry
Rollout	Blue/green, canary, automated analysis	Warm standby, shadow process, host-by-host or small cohort
Hot path	General service mesh/telemetry where acceptable	Minimal dependencies; lock-free/zero-allocation as justified; off-path telemetry
State	External durable store or replicated service	Event log/checkpoint, local optimized state with authoritative recovery
Performance evidence	Service latency/throughput/resource budget	Tail latency, jitter, CPU/NUMA/cache/network/-clock profile
Shared controls	Immutable artifacts, independent risk, activation envelope, audit, rollback, SLOs	Same

8.4 Progressive rollout and rollback

Rollout starts with replay and synthetic probes, then paper, shadow, bounded canary, and live. Canary dimensions may be capital, instruments, venues, time window, order size, participation, or strategy allocation. Guardrails must be measurable and connected to automatic disable or operator escalation.

Rollback is multi-dimensional: binary, model, feature graph, configuration, limit set, schema compatibility, and state. An old binary is not a valid rollback if it cannot read the current state or schema. Migrations therefore use expand-migrate-contract, dual-read/write where needed, and explicit backward-compatibility windows.

8.5 Disaster recovery

Each service tier has RTO and RPO. Recovery tests cover registry, configuration, state, audit, market/reference data, OMS, risk, and key material. Backups are useful only after a restore is verified. T0 recovery also requires reconciliation with venue, broker, and clearing records before reactivation.

Chapter 9

Monitoring, SLOs, and incident control

9.1 Production control loop

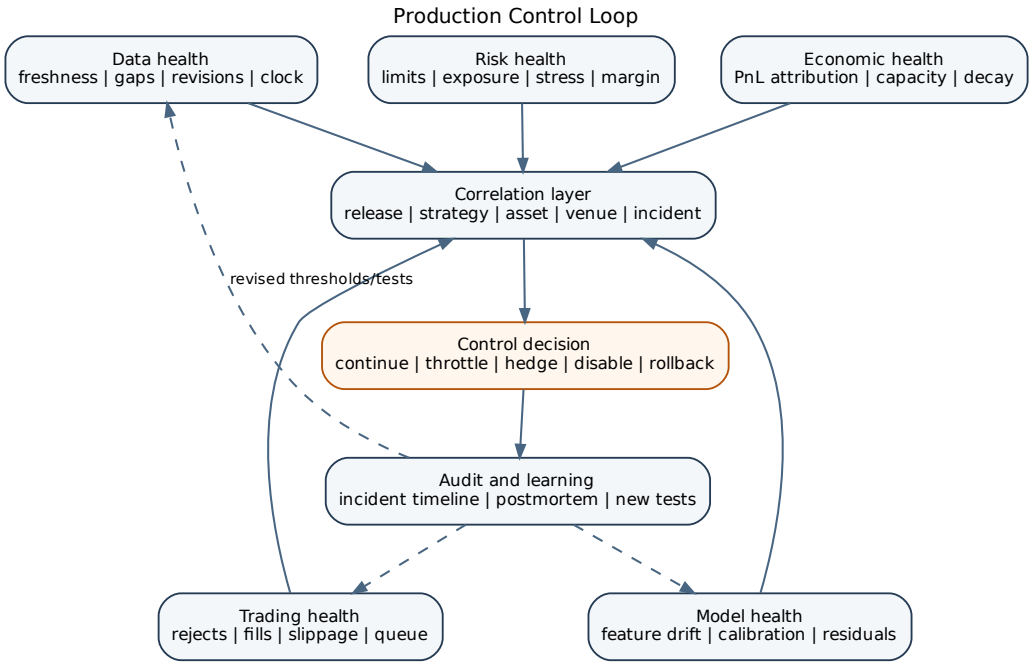


Figure 9.1: Technical and economic monitoring converge on explicit control actions.

OpenTelemetry provides vendor-neutral traces, metrics, logs, and context propagation [24]. Prometheus guidance emphasizes metric types and naming, while Google SRE recommends SLOs with error budgets rather than unrealistic 100-percent targets [25, 26]. Progressive delivery systems such as Argo Rollouts illustrate automated canary/blue-green analysis and rollback patterns [27].

9.2 Monitoring planes

Plane	Signals	Control actions
Platform	Availability, error rate, tail latency, throughput, saturation, queue depth, memory, clock, recovery	Restart, failover, capacity add, release rollback, traffic shift.
Data	Freshness, gaps, duplicates, schema, revisions, cross-source breaks, PIT violations, entitlement	Quarantine, source failover, stale-data mode, stop affected strategies, backfill.
Feature/model	Missingness, drift, correlation, calibration, residuals, score distribution, inference latency, state divergence	Alert/research review, conservative fallback, reduce scope, disable model, retrain through controlled promotion.

Trading/execution	Decision/order rate, rejects, cancels, fills, slippage, queue, adverse selection, venue/session health	Throttle, reroute, cancel, hedge-only, venue kill, strategy kill.
Risk	Position, exposure, limit utilization, stress, margin, liquidity, drawdown, reconciliation	Block new risk, reduce, hedge, kill, escalate.
Economic	Gross/net PnL, alpha capture, spread, fees, impact, timing, funding, borrow, hedge/FX residual, capacity decay	Research recalibration, cost update, allocation reduction, retirement review.

9.3 SLO design

SLOs are strategy-horizon and tier specific. Market-data freshness for a microsecond market maker and a daily cross-sectional strategy cannot share a threshold. A useful design relates each budget to the decision horizon and economic loss. Track multi-window error-budget burn:

$$B_w = \frac{\text{bad events in window } w}{(1 - \mathrm{SLO}) \times \text{eligible events in } w}. \tag{9.1}$$

Fast and slow burn alerts distinguish acute failure from persistent degradation.

9.4 PnL and TCA attribution

A minimum daily and intraday decomposition is

$$\Pi_{\mathrm{net}} = \Pi_{\mathrm{signal}} - C_{\mathrm{spread}} - C_{\mathrm{fees}} - C_{\mathrm{impact}} - C_{\mathrm{timing}} - C_{\mathrm{funding}} - C_{\mathrm{borrow}} + \Pi_{\mathrm{hedge}} + \Pi_{\mathrm{FX}} + \varepsilon. \tag{9.2}$$

The residual ε is monitored and reconciled. Attribution is keyed to release, strategy, model, asset, venue, account, and order cohort so a model improvement is not confused with cheaper execution or changing risk.

9.5 Alert design

Every actionable alert has severity, owner, SLO/control objective, threshold rationale, deduplication key, runbook, safe action, escalation, and closure evidence. Alerts without an owner or action should be dashboards, not pages. P0/P1 scenarios include uncontrolled order risk, kill-switch failure, position divergence, critical stale data, lost venue state, compromised key, and inability to reconcile.

9.6 Incident lifecycle

Detect, contain, preserve evidence, reconcile positions/orders, communicate, recover, verify, and learn. Postmortems are blameless but exact: timeline, release, data, states, control behavior, economic impact, contributing factors, and why existing tests/alerts failed. Corrective actions add automated tests or controls, not only prose.

Chapter 10

Security, model risk, and regulatory alignment

10.1 Secure software supply chain

NIST's Secure Software Development Framework organizes practices for secure development [20]. SLSA defines provenance and increasing build assurance, SPDX is an international SBOM standard, and Sigstore/Cosign supports signing and verifying artifacts and attestations [21, 22, 23]. The institutional minimum is pinned dependencies, hermetic hosted builds, signed provenance, SBOM, digest deployment, policy verification, and protected source with independent review.

10.2 Access and separation of duties

Human production access is read-only by default. Write or activation access is just-in-time, time-bounded, authenticated, authorized, logged, and subject to dual control for material actions. Service identities are workload-specific and short-lived. Research credentials cannot place orders. The strategy cannot change its own risk envelope.

10.3 Algorithmic trading controls

The FCA's 2025 review highlights governance, testing including simulation and stress scenarios, and continuous monitoring for algorithmic trading [17]. SEC Rule 15c3-5 requires broker-dealers with market access to maintain financial and regulatory risk controls under their direct and exclusive control, with review and certification requirements [18]. FIX Trading Community publishes multi-asset standards and guidance for algorithm identity, parameters, testing, and certification [19]. These sources reinforce the architecture's independent pre-trade risk, controlled testing, auditability, and ongoing monitoring.

10.4 Model-risk operating principles

The platform should classify model materiality by inherent risk, exposure, purpose, complexity, interconnectedness, and change sensitivity. Validation is proportionate but independent. It covers conceptual soundness, data, implementation, outcomes, limitations, sensitivity, benchmark comparisons, OOS/OOT tests, ongoing monitoring, change control, and retirement. Simple deterministic rules may not be models under every formal definition, but they still require software, risk, and change controls.

10.5 Control evidence map

Control objective	Primary implementation	Evidence source
Reviewed change	Protected branch, CODEOWNERS, required approvals, merge train	MR approvals, SHA-bound pipeline, reviewer identities.
Causal research	Bitemporal data, split library, trial ledger	Dataset manifest, leakage suite, experiment record.
Validated model	Model card, independent validation, findings workflow	Validation report, approvals, limitations, monitoring plan.
Controlled market access	External risk gateway, limits, kill switches, dual control	Risk decision logs, limit versions, game-day evidence.
Reproducible release	Hermetic build, provenance, SBOM, signed immutable manifest	Artifact digest, attestation, registry entry.
Ongoing monitoring	Correlated telemetry, SLOs, data/model/TCA/risk dashboards	Alert tests, dashboards, incident records.
Resilience	State recovery, reconciliation, backup/restore, failover	Scenario results, RTO/RPO tests, reconciliation reports.
Audit and accountability	Immutable audit events, JIT access, session/action identity	Audit stream, access reviews, evidence retention.

Chapter 11

Implementation roadmap: 96 merge requests

11.1 How to use the roadmap

Each row is a logical MR package. If a row would exceed reviewable size, split it into code, migration, backfill, and rollout MRs while preserving the ID as an epic. Every MR inherits the quality gates and reviewers from its risk class. A wave is complete only when its exit criteria are demonstrated in the target environment, not when all code is merged.

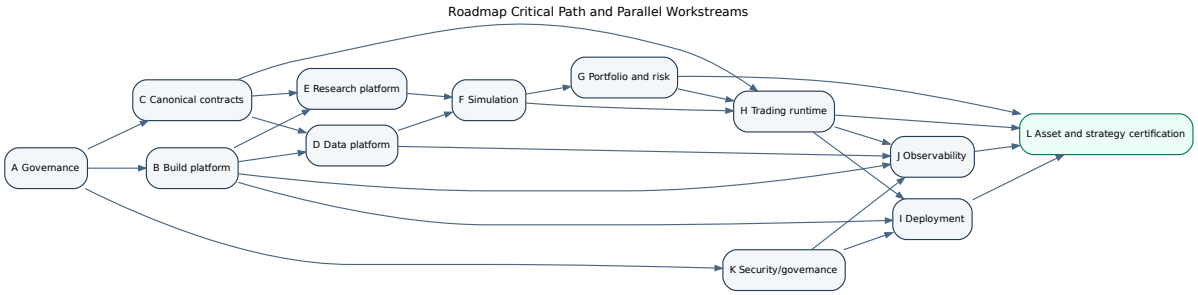


Figure 11.1: Critical path and parallel workstreams.

11.2 Illustrative milestones

Milestone	Waves	Outcome	Do not proceed until
M0 - Governed source	A–B	Protected monorepo, hermetic build, evidence pipeline, signed artifacts	Ownership, risk classes, build reproducibility, and supply-chain controls are live.
M1 - Causal research	C–E	Canonical contracts, point-in-time data, experiment ledger, robust evaluation	Leakage suite, snapshot reproducibility, and independent research report pass.
M2 - Executable simulation	F–G	Deterministic market/execution simulation, costs, portfolio, independent risk	Replay parity, accounting, cost calibration, risk and stress scenarios pass.
M3 - Safe production path	H–K	OMS/adapters, paper/shadow, immutable deployment, SLOs, security, model governance	State recovery, kill switch, shadow, rollback, DR, and audit evidence pass.
M4 - Capital scale	L	Asset and strategy certification with controlled canary and scale-up	No critical finding; live results reconcile; capacity and operational review approve each increase.

11.3 Detailed MR sequence

ID	MR / objective	Principal artifacts	Exit evidence and quality gates	Depends
A01	Repository charter and monorepo boundary	Create the repository charter, scope, non-goals, naming conventions, protected main branch, and a map of systems that remain outside the monorepo. Define that a monorepo is a source and governance boundary, not one deployable or one failure domain.	Charter approved by platform, quantitative research, trading engineering, risk, SRE, security, and compliance; all existing repositories classified as migrate, bridge, or retain; no direct push to main. QG13, QG12.	-
A02	Architecture principles and ADR framework	Add architecture principles, ADR template, decision log, dependency rules, deprecation policy, and exception process. Record initial decisions for build tooling, languages, schemas, storage, runtime lanes, deployment, and observability.	Every cross-cutting decision has an owner, alternatives, consequences, review date, and reversal plan; ADR lint and link checking run in CI. QG13.	A01
A03	Domain ownership and CODEOWNERS	Define ownership for schemas, data, research, simulation, portfolio, risk, trading, platform, security, and asset adapters. Encode independent review requirements and escalation paths.	Protected paths require Code Owner approval; author cannot satisfy independent validation; orphaned paths are rejected by policy. QG12, QG13.	A01-A02
A04	Change risk classification	Implement MR classes R0 through R4 based on capital-at-risk, blast radius, data/model materiality, security privilege, and reversibility. Map each class to mandatory gates, reviewers, rollout modes, and evidence retention.	Classifier is deterministic and policy-as-code; sample MRs are correctly classified; R4 cannot merge with a single approver or a waived hard control. QG12, QG15.	A02-A03
A05	MR template and evidence bundle	Create MR template fields for problem, hypothesis, scope, risk class, data/model lineage, test plan, benchmark delta, rollout, rollback, monitoring, runbook, approvals, and post-deploy checks. Define machine-readable evidence manifest.	MR linter blocks missing required fields; evidence manifest is tied to the commit SHA; example bundles validate end-to-end. QG13, QG15.	A04
A06	Data, model, strategy, service, and control inventories	Define inventories and stable identifiers for datasets, models, strategies, services, limits, venues, and releases. Add ownership, criticality, dependencies, retention, and status fields.	Inventories are queryable, versioned, and linked to deployment evidence; stale ownership and missing lineage generate findings. QG05, QG14.	A01-A05
A07	Threat model and regulatory control map	Document trust boundaries, abuse cases, privileged actions, data entitlements, supply-chain threats, market-abuse risks, model-risk controls, and jurisdictional mapping.	Threat model reviewed by security and trading risk; controls have testable owners and evidence sources; regulatory mapping is explicitly jurisdiction-tailored and not treated as legal advice. QG12, QG13.	A02,A06
A08	Program dependency graph and baseline KPIs	Add roadmap DAG, delivery dashboards, baseline build time, test time, deployment lead time, incident rate, data-quality defects, research reproducibility rate, and rollback performance.	Metrics are collected automatically; no vanity metric is used as a substitute for hard gates; target ranges and owners are approved. QG14.	A01-A07

ID	MR / objective	Principal artifacts	Exit evidence and quality gates	Depends
B01	Repository skeleton and dependency constraints	Create top-level packages, ownership files, documentation structure, test areas, environment overlays, and automated dependency-boundary checks.	A clean checkout builds; forbidden upward or cyclic dependencies fail CI; generated code is separated and reproducible. QG01, QG02.	A02-A05
B02	Polyglot hermetic build toolchains	Configure Bazel toolchains for Python, C++ and/or Rust, Protobuf, SQL generation, and documentation. Pin compilers, linkers, interpreters, and system libraries.	Local and CI builds produce identical digests for a representative target set; undeclared inputs are caught by sandboxing. QG02, QG10.	B01
B03	Hermetic developer environments	Provide Nix flake or equivalent devcontainers, pre-commit hooks, IDE integration, and bootstrap scripts that contain no hidden workstation dependency.	New machine bootstrap succeeds from documented prerequisites; environment lock is reviewed; rollback to a prior toolchain is demonstrated. QG02, QG13.	B02
B04	Language standards and static quality tooling	Adopt formatters, linters, type checking, sanitizers, warnings-as-errors policy, API stability rules, numeric tolerance standards, and code generation conventions.	Representative defects are rejected; suppression requires owner, rationale, and expiry; coverage and mutation-test baselines are recorded. QG01, QG03, QG04.	B02-B03
B05	CI fast lane and affected-target analysis	Implement preflight, hermetic build, unit tests, schema checks, secrets scan, and affected-target selection. Separate generated artifacts from source diffs.	Median fast-lane target is under the agreed budget; target selection is validated against full builds; failure is fail-closed. QG01-QG05, QG12.	B01-B04
B06	Remote cache, execution, sharding, and flake control	Add authenticated remote cache/execution, test sharding, retries only for infrastructure faults, quarantine workflow, and flake ownership.	Cache poisoning protections are tested; flaky tests cannot silently pass; full-suite equivalence and deterministic test seeds are demonstrated. QG02, QG03, QG11.	B05
B07	Dependency, license, and approved-registry policy	Pin direct and transitive dependencies, restrict package sources, scan licenses and vulnerabilities, define upgrade cadence, and prohibit runtime package installation.	Lockfiles are complete; unapproved dependency or mutable reference fails CI; emergency override is time-bounded and audited. QG12.	B02-B06
B08	Artifact provenance, SBOM, signing, and verification	Emit build provenance, SPDX-compatible SBOM, checksums, test evidence, and signed attestations for binaries, containers, models, and configuration bundles. Enforce digest-based deployment.	Admission policy rejects unsigned, unprovenanced, mutable-tag, or policy-noncompliant artifacts; verification and key-compromise drills pass. QG02, QG12, QG15.	B07,A07
C01	Stable instrument identity and symbology mappings	Define internal InstrumentId and validity-dated mappings to vendor, venue, ISIN/CUSIP/SEDOL, ticker, contract, and chain identifiers. Separate economic instrument, listing, contract, and venue symbol.	Round-trip and ambiguity tests pass; symbol reuse, delisting, relisting, and vendor corrections are covered by golden fixtures. QG03-QG05.	B01-B05
C02	Venue, session, calendar, and market-status contracts	Define venue identifiers, MICs, trading sessions, auctions, holidays, half-days, halts, limit states, settlement calendars, and timezone handling.	Cross-timezone and daylight-saving tests pass; no runtime logic depends on host-local time; calendar changes are versioned. QG03-QG05.	C01
C03	Typed units, currencies, prices, quantities, and conventions	Create typed money, FX, price, yield, volatility, rate, duration, multiplier, lot, tick, and quantity abstractions. Encode rounding and dimensional checks.	Invalid unit arithmetic fails at compile or validation time; tick/lot rounding and currency conversion have property tests and audited tolerances. QG03, QG04.	C01-C02

ID	MR / objective	Principal artifacts	Exit evidence and quality gates	Depends
C04	Event-time, knowledge-time, ingestion-time, and bitemporal model	Specify timestamps, sequence numbers, source revision, valid-from/to, as-of semantics, clock quality, and causal query rules.	A decision-time query cannot return data whose knowledge-time exceeds the decision time; late and corrected records are replayable. QG04, QG05.	C01-C03
C05	Market, reference, and lifecycle event schemas	Define versioned schemas for trades, quotes, book deltas, auction events, rates, curves, corporate actions, dividends, rolls, fixings, funding, borrow, and instrument lifecycle.	Backward/forward compatibility matrix passes; unknown fields are handled safely; sample data across all target asset classes validates. QG03, QG05.	C01-C04
C06	Order, execution, and fill state machines	Define OrderIntent, parent/child order, acknowledgement, reject, amend, cancel, partial fill, bust/correct, allocation, and terminal states with idempotency keys.	Formal transition table has no impossible or ambiguous state; property tests cover duplicate, delayed, and out-of-order messages. QG03, QG04, QG09.	C01-C05
C07	Position, cash, PnL, exposure, and limit schemas	Define positions/lots, cash ledgers, realized/unrealized PnL, fees, funding, borrow, FX translation, exposures, risk snapshots, and limit decisions.	Accounting conservation and sign-convention tests pass; schema supports intraday and end-of-day reconciliation without lossy conversion. QG03-QG05, QG08.	C03,C05-C06
C08	Schema registry, compatibility policy, and golden fixtures	Implement schema registry, generated clients, compatibility checks, fixture catalog, migration workflow, and deprecation windows.	Breaking changes require expand-migrate-contract plan; every schema has owner, examples, semantic version, and replay tests. QG02-QG05, QG13.	C01-C07
D01	Immutable raw landing with integrity and entitlements	Land vendor payloads and files unchanged with checksums, source metadata, license/entitlement tags, arrival time, retention class, and encryption.	Object mutation is prevented; sampling proves checksum integrity; unauthorized dataset access is denied and logged. QG05, QG12.	B08,C04
D02	Connector SDK with checkpointing and replay	Create connector interfaces for streams, files, APIs, and databases with retries, backpressure, sequence tracking, idempotency, dead-letter handling, and replay.	Injected disconnects, duplicates, sequence gaps, and partial files recover without data loss or double application. QG03, QG09, QG11.	D01,C05
D03	Canonical normalization and cross-source reconciliation	Normalize raw payloads into canonical contracts; retain source payload pointer and transformation version; reconcile redundant sources where available.	Golden transformations are deterministic; source-to-canonical lineage is complete; discrepancies exceed thresholds only with an owned exception. QG02, QG05.	D01-D02, C08
D04	Reference and master data service	Build mastered symbology, calendars, contract specifications, corporate actions, FX/rates, and vendor-priority rules with temporal validity.	Known edge cases pass; no strategy embeds private symbol maps or calendars; changes emit impact reports and replay deltas. QG05, QG09.	C01-C05,D03
D05	Data quality framework and dataset SLOs	Implement contract checks for freshness, completeness, uniqueness, gaps, monotonicity, bounds, distribution shifts, cross-field invariants, and source reconciliation.	Checks run at ingestion and publication; severity maps to quarantine, degrade, or stop; dashboards and runbooks exist. QG05, QG14.	D02-D04
D06	Point-in-time dataset builder and snapshot semantics	Build as-of joins, revision-aware snapshots, universe membership, survivorship controls, corporate-action availability, and reproducible dataset fingerprints.	Leakage attack suite fails unsafe queries; same manifest yields same dataset digest; snapshot retrieval meets performance targets. QG02, QG05, QG10.	C04,D03-D05
D07	Lineage, catalog, entitlements, and usage accounting	Integrate dataset catalog, owners, descriptions, lineage graph, quality status, entitlements, cost attribution, and retention.	Every promoted feature/model traces to raw sources and transformations; orphaned or unentitled lineage blocks promotion. QG05, QG12-QG14.	D01-D06,A06

ID	MR / objective	Principal artifacts	Exit evidence and quality gates	Depends
D08	Replay, correction, backfill, and reconciliation service	Provide deterministic historical replay, correction notices, backfill orchestration, downstream impact analysis, and end-to-end reconciliation.	A corrected source record can identify affected features, backtests, models, and releases; backfill is idempotent and audited. QG05, QG09, QG11.	D06-D07
E01	Research SDK and explicit runtime context	Create typed APIs for data access, clocks, universes, features, labels, models, portfolio policies, costs, and reporting. Ban implicit global state and workstation paths.	Example research runs in notebook, batch, and CI with identical semantics; imports respect dependency boundaries. QG02-QG04, QG13.	B,C,D06
E02	Experiment and committed trial ledger	Record hypothesis, economic mechanism, author, code SHA, environment, data snapshot, label, horizon, universe, features, hyperparameters, seeds, benchmark, and every material trial including failures.	Trial identifiers are immutable; selective deletion is prevented; promotion report includes effective trial count and rejected variants. QG05, QG06, QG13.	E01,A06
E03	Dataset, feature, label, and universe APIs	Implement versioned declarative definitions, as-of materialization, missing-value policy, neutralization, transformations, label overlap metadata, and universe locks.	Offline values reproduce exactly; feature/label knowledge-time checks pass; no fit operation uses validation or OOS data. QG04-QG06.	D06-D08, E01-E02
E04	Time-series split, purging, embargo, and walk-forward library	Provide blocked, rolling, expanding, purged and embargoed splits; cross-sectional and regime partitions; nested model selection; final untouched OOS.	Synthetic leakage tests detect overlap; split manifest is persisted; OOS cannot be queried before promotion decision without recorded exception. QG05, QG06.	E03
E05	Baseline, model, calibration, and ensemble adapters	Define common fit/predict/update interfaces for linear, tree, neural, Bayesian, time-series, and rule-based models; include deterministic baselines and calibration.	Every candidate beats declared baselines on pre-specified metrics or documents why; serialization and inference parity tests pass. QG03, QG04, QG06.	E03-E04
E06	Evaluation, robustness, and multiple-testing controls	Implement signed bias, MAE/Huber/quantile metrics, ranking and calibration metrics, turnover, exposure, regime stability, sensitivity, ablation, Deflated Sharpe Ratio, PBO or equivalent, and FDR controls.	Metric definitions are versioned; confidence intervals use dependence-aware methods where needed; trial-adjusted evidence is mandatory for promotion. QG06.	E02-E05
E07	Reproducible distributed runner and resource governance	Run experiments on local, cluster, and cloud/HPC backends with pinned inputs, deterministic seeds, resource quotas, checkpointing, caching, and cost accounting.	Same manifest reproduces within declared numeric tolerance; interrupted jobs resume correctly; runaway jobs are bounded. QG02, QG04, QG10-QG12.	E01-E06,B06
E08	Model registry, research report, and promotion card	Register model, feature graph, dataset, code, environment, cost model, limits, validation, intended use, limitations, monitoring, and sign-offs. Use aliases only as controlled pointers.	No candidate can become shadow or production without complete card, independent validation, immutable digests, and linked evidence. QG06-QG08, QG13-QG15.	E02-E07,B08
F01	Deterministic event-driven simulation engine	Implement ordered event queue, market/strategy/risk/order events, deterministic scheduler, replayable state, and extensible asset/venue models.	Event ordering is specified; repeat runs match bitwise or within declared tolerance; throughput benchmark is recorded. QG02-QG04, QG10.	C,D08,E01
F02	Clock, randomness, state snapshot, and causality controls	Inject simulation clock, seeded random streams by component, checkpoint/restore, and no-lookahead guards. Separate exchange, receipt, decision, send, and fill times.	Changing thread count does not change results; restart from checkpoint matches uninterrupted run; future data access raises a hard error. QG04-QG05, QG11.	F01,C04

ID	MR / objective	Principal artifacts	Exit evidence and quality gates	Depends
F03	Accounting, corporate-action, expiry, and roll engine	Apply cash movements, dividends, splits, coupons, accruals, exercise/assignment, expiries, contract rolls, FX translation, and fees to the ledger.	Accounting conservation holds under golden scenarios; broker statements can be reconciled at lot and cash level. QG03-QG05, QG09.	F01-F02,C07, D04
F04	Market-state and order-book simulator	Model quotes, trades, L1/L2/L3 where available, auctions, halts, price limits, session transitions, and stale/locked/crossed markets.	Replay reproduces observed market state; invalid books are quarantined; asset-specific scenario tests pass. QG03-QG05, QG09.	F01-F03, D03-D05
F05	OMS, fill, queue, and execution simulator	Simulate acknowledgements, rejects, cancels, amendments, partial fills, queue position, venue throttles, outages, busts/corrections, and drop-copy.	Calibrated fill distributions are compared with live or vendor benchmarks; idempotency and state-machine invariants hold. QG04, QG09.	C06,F04
F06	Transaction cost, impact, borrow, funding, and capacity models	Implement spread, fees/rebates, latency, adverse selection, temporary/permanent impact, borrow availability/rate, financing, margin, and capacity curves.	Costs are calibrated by asset, venue, order type, horizon, and participation; sensitivity and conservative stress bands are in reports. QG06-QG08.	F03-F05,D07
F07	Multi-venue latency and scenario framework	Support venue-specific latencies, routing, fragmented liquidity, failure scenarios, regime shocks, data degradation, and counterfactual policy tests.	Scenario catalog covers normal, stressed, and pathological cases; results are reproducible and tied to release evidence. QG09-QG11.	F04-F06
F08	Replay parity, differential tests, golden strategies, and performance	Create minimal strategies with analytically known results; compare research, simulation, paper, and live decision traces; benchmark throughput, memory, and tail latency.	No unexplained parity delta; performance regressions above budget block merge; golden outputs are reviewed and versioned. QG03-QG04, QG09-QG10.	F01-F07
G01	Signal-to-target portfolio API	Define forecast, confidence, horizon, target position, rebalance intent, and execution urgency contracts. Separate alpha, portfolio construction, and execution policy.	Reference strategies demonstrate independent substitution of each stage; units and timing are explicit. QG03-QG06.	E05,F08,C07
G02	Position, exposure, and inventory service	Provide authoritative intraday positions, cash, pending orders, inventory, factor/sector/currency/greek exposures, and staleness state.	Reconciles with OMS and clearing views; stale or divergent state causes documented fail-safe behavior. QG05, QG08-QG09, QG14.	C07,F03,G01
G03	Risk model and covariance interface	Define versioned factor models, covariance forecasts, specific risk, scenario shocks, liquidity risk, and model-quality metadata across asset classes.	Numerical conditioning, PSD repair, unit scaling, missing-exposure behavior, and benchmark comparisons pass. QG04, QG06, QG08.	G02,E06
G04	Optimizer and constraint library	Implement convex and approved non-convex optimizers, turnover/transaction-cost penalties, exposure, concentration, liquidity, borrow, leverage, and cardinality constraints.	KKT or solver diagnostics are captured; infeasible cases return explainable fallbacks; gradient and perturbation tests pass. QG04, QG07-QG08.	G01-G03,F06
G05	Independent pre-trade risk gateway	Create external-to-strategy checks for notional, price collars, fat-finger, position, exposure, credit, message rate, locate/borrow, venue, and regulatory constraints. Default fail-closed.	Risk decisions are deterministic, low-latency, logged, and replayable; bypass requires dual-control break-glass; timeout policy is tested. QG08-QG12.	C06-C07, G02-G04

ID	MR / objective	Principal artifacts	Exit evidence and quality gates	Depends
G06	Intraday limits, stress, liquidity, margin, and collateral	Implement hierarchical limits, stress scenarios, drawdown controls, liquidity horizons, margin/collateral consumption, and concentration aggregation.	Limit aggregation reconciles across strategies and accounts; stressed conditions and partial outages are exercised. QG08-QG11, QG14.	G03-G05
G07	Kill switches, throttles, and degraded modes	Implement global, strategy, asset, venue, account, and instrument kill switches; order cancel workflows; throttle and hedge-only modes; independent control channel.	Game day proves activation under data loss and process failure; kill action is idempotent and observable; recovery requires explicit authorization. QG08-QG11, QG15.	G05-G06, H04-prep
G08	Risk, PnL, and control reconciliation evidence	Build independent recomputation, limit-decision replay, PnL attribution, exception queue, sign-off workflow, and evidence retention.	Daily and intraday breaks are owned and aged; material breaks block scale-up; reconciliation inputs are independent where feasible. QG05, QG08-QG09, QG13-QG14.	G02-G07,F06
H01	Shared strategy kernel for research and live	Define pure strategy logic, typed state, event handlers, feature/signal/target interfaces, and deterministic serialization. Runtime resources are injected.	Same kernel runs in replay, paper, shadow, and live; notebook-only code cannot enter production package. QG02-QG04, QG09.	E01,G01,F08
H02	Live runtime, scheduling, and dependency injection	Implement process lifecycle, event subscriptions, clocks, state stores, configuration, telemetry, health checks, and resource limits for standard and low-latency lanes.	Startup fails on incompatible schema/config; liveness is not confused with readiness; bounded backpressure behavior passes. QG09-QG11, QG14.	H01,B08, I-prep
H03	State persistence, recovery, and idempotency	Add write-ahead or event-sourced state, checkpoints, replay, idempotency keys, duplicate suppression, and ownership fencing.	Crash/restart and split-brain scenarios do not duplicate orders or lose authoritative state; recovery time meets tier target. QG03-QG04, QG09-QG11.	H02,C06
H04	Production OMS state machine	Build parent/child lifecycle, amend/cancel, partial fills, allocations, drop-copy comparison, order aging, and operator controls.	Certification suite covers all transitions and venue anomalies; state reconciles after restart; manual actions are authenticated and audited. QG03-QG04, QG08-QG12.	C06,H03,G05
H05	Execution adapter and FIX/session SDK	Provide adapter contract, FIX or native session layer, sequence/recovery, throttles, heartbeats, encoding, time sync, and conformance harness.	Protocol transcripts pass certification; replay and disconnect recovery are deterministic; secrets are externalized. QG09-QG12.	H04,C08
H06	Reference venue adapter and independent drop-copy	Implement one production-grade venue adapter, market data/order entry separation, drop-copy ingestion, and broker/exchange reconciliation.	Venue certification, reject scenarios, bust/correct, session reset, and recovery tests pass in test environment; no unowned exception. QG08-QG12.	H05,D02, G08
H07	Paper broker and shadow router	Provide paper execution using live data and shadow decisions with order suppression, counterfactual fills, trace comparison, and safety interlocks.	Shadow mode cannot emit executable messages; decision parity, latency, data freshness, and risk decisions are monitored for an agreed observation window. QG09-QG15.	H01-H06, J-prep
H08	Canary and live activation certification	Add limited-universe/capital activation, dual-control enablement, guardrails, automatic rollback triggers, operator checklist, and post-activation validation.	All R4 evidence is complete; canary breaches disable safely; capital and scope cannot exceed signed envelope; rollback is rehearsed. QG08-QG15.	H07,G07,I,J, K

ID	MR / objective	Principal artifacts	Exit evidence and quality gates	Depends
I01	Release manifest and immutable artifact registry	Define release as code SHA, build provenance, binary/-container digest, model, feature graph, data snapshot references, config, schemas, limits, tests, approvals, and runbooks.	Registry rejects incomplete or mutable manifests; every live process reports exact release identity. QG02, QG12-QG15.	B08,E08,C08
I02	Environment promotion and separation policy	Implement development, CI, replay, staging, paper, shadow, canary, and production environments with one-way artifact promotion and separate credentials.	No rebuild occurs between stages; prod cannot pull from developer registries; environment diff report is generated. QG02, QG12, QG15.	I01,A04
I03	Configuration versioning, validation, and feature flags	Create typed configuration schemas, inheritance limits, secret references, validation, diffing, staged flags, expiry, and ownership.	Unknown or out-of-range config fails before activation; config digest is in release manifest; stale flags generate findings. QG03-QG05, QG13-QG15.	I01-I02,C08
I04	Service identities, secrets, and privileged control plane	Integrate short-lived workload identities, secret manager, key rotation, operator authentication, authorization, and audited break-glass.	No static prod secret in repo/image; compromised credential drill and rotation pass; privilege is least and time-bounded. QG12.	A07,B08,I02
I05	GitOps deployment lane for standard services	Define declarative manifests, policy admission, rollout health, capacity, autoscaling, disruption budgets, and environment overlays for non-ultra-low-latency services.	Only signed manifests deploy; drift is detected and reverted or approved; rollback to prior digest is automated. QG11-QG15.	I01-I04,H02
I06	Low-latency bare-metal deployment lane	Provide immutable packages, host attestation, NUMA/CPU/network configuration, PTP/NTP health, staged host groups, warm standby, and same promotion API as GitOps lane.	Host drift and clock degradation block activation; benchmark and failover pass; no ad hoc file copy or mutable host state. QG10-QG15.	I01-I04,H02, H05
I07	Progressive rollout, rollback, and migration orchestration	Implement blue/green or canary for services, shadow/canary for strategies, expand-migrate-contract for schemas, dual-read/write, and coordinated rollback.	Rollback includes binary, model, config, schema compatibility, and state; partial deployment and failed migration scenarios recover. QG09-QG11, QG15.	I05-I06,C08
I08	Backup, restore, disaster recovery, and break-glass	Define RTO/RPO by tier; back up registries, state, configs, audit, and critical data; run restore, regional/site failover, and break-glass exercises.	Restores are verified, not assumed; recovery evidence is retained; emergency changes require post-incident review and compensating controls. QG11-QG15.	I01-I07,G07
J01	OpenTelemetry-based telemetry substrate	Standardize traces, metrics, logs, resource identity, context propagation, sampling, collectors, and retention tiers. Separate control telemetry from latency-sensitive hot paths where needed.	A release, strategy, account, asset, venue, order, and incident can be correlated end-to-end; telemetry loss is itself monitored. QG14.	B01,H02,I01
J02	Metric, log, trace, and cardinality conventions	Define naming, units, labels, histogram buckets, event schemas, sensitive-data handling, cardinality budgets, and clock-skew fields.	Lint rejects unbounded labels and unit ambiguity; representative dashboards remain performant under load. QG10, QG12, QG14.	J01,C03-C04
J03	Platform SLIs, SLOs, error budgets, and release policy	Set tiered availability, latency, freshness, correctness, recovery, and durability indicators; define multi-window burn alerts and deployment freeze criteria.	SLOs are tied to trading impact; error-budget policy is exercised; no 100-percent target hides operational trade-offs. QG14-QG15.	J01-J02, A03-A04

ID	MR / objective	Principal artifacts	Exit evidence and quality gates	Depends
J04	Data freshness, quality, lineage, and clock monitors	Monitor source lag, gaps, duplicates, revisions, schema drift, entitlement failures, PIT violations, cross-source breaks, and time synchronization.	Alerts identify affected strategies/releases; stale data triggers documented degrade/stop policy; backfill closes incidents. QG05, QG11, QG14.	D05-D08, J01-J03
J05	Feature, model, strategy, and regime monitoring	Track missingness, distribution and correlation drift, prediction drift, calibration, residuals, turnover, exposures, decision rate, state divergence, and regime indicators.	Thresholds have statistical and economic rationale; delayed labels are handled; monitoring does not auto-disable solely on noisy PnL without guardrails. QG06, QG08, QG14.	E08,H01, J01-J04
J06	Execution quality, TCA, capacity, and PnL attribution	Compute spread capture, implementation shortfall, timing, impact, adverse selection, fill/reject ratios, queue metrics, fees, borrow, funding, hedge and FX attribution.	Simulated versus realized cost gaps are tracked; material model misspecification opens owned research and control actions. QG07-QG09, QG14.	F06,H06, G08,J01-J05
J07	Alert routing, runbooks, audit event stream, and operator UX	Map severity to owner and escalation; link every actionable alert to runbook; capture limit, config, model, deployment, manual order, and kill-switch events in immutable audit stream.	Alert tests page the correct rotation; duplicate/noisy alerts have budgets; all privileged actions are attributable. QG12-QG14.	J03-J06,A06
J08	Synthetic probes, game days, incident response, and postmortems	Run synthetic market/order flows, venue/data failover, kill-switch, restore, clock, capacity, and degraded-mode exercises. Feed postmortem actions into tests and roadmap.	Exercises meet tier frequency; corrective actions have owners and due dates; recurrence is measured. QG09-QG15.	J04-J07,I08, G07
K01	RBAC, JIT access, separation of duties, and production access	Define human and service roles, just-in-time elevation, session recording, two-person controls for material actions, and periodic access review.	Default prod access is read-only; dormant/excess privileges are removed; emergency access is time-bounded and reviewed. QG12.	A03-A07,I04
K02	Network segmentation and egress policy	Separate research, data, control, trading, risk, and management networks; restrict egress, vendor endpoints, lateral movement, and admin paths.	Policy tests and attack simulation show intended isolation; low-latency exceptions are documented and compensated. QG11-QG12.	K01,I05-I06
K03	Software supply-chain policy enforcement	Enforce approved source, reviewed commits, hermetic build, provenance, SBOM, signature verification, digest pinning, and admission policy against artifacts and dependencies.	Tampered artifact, forged provenance, mutable tag, and unapproved package tests are blocked. QG02, QG12, QG15.	B07-B08,I01
K04	Secrets, keys, encryption, and rotation	Standardize encryption in transit/at rest, key ownership, hardware/KMS use where appropriate, secret leasing, rotation, revocation, and backup-key controls.	Rotation does not interrupt trading beyond budget; revoked keys cannot authenticate; key inventory is complete. QG11-QG12.	I04,K01-K03
K05	Vulnerability, license, dependency, and patch operations	Implement severity/SLA policy, exploitability triage, exception expiry, coordinated patch testing, license obligations, and end-of-life tracking.	Critical exposure has owned disposition; patch does not bypass simulation/performance controls; dependency inventory reconciles to SBOM. QG10-QG12.	B07,K03
K06	Data entitlement, privacy, retention, and deletion controls	Enforce vendor terms, geographic/user restrictions, need-to-know access, sensitive fields, retention, deletion, and derived-data obligations.	Entitlement follows lineage into derived data; access and deletion tests pass; promotion blocks unclear rights. QG05, QG12-QG13.	D01,D07,K01

ID	MR / objective	Principal artifacts	Exit evidence and quality gates	Depends
K07	Model-risk governance, independent validation, and monitoring	Implement materiality tiers, model inventory, intended use, effective challenge, conceptual soundness, OOS/OOT analysis, limitations, change control, validation, and ongoing monitoring.	Material models cannot self-validate; validation scope is risk-based; findings, limitations, compensating controls, and retirement are tracked. QG06, QG08, QG13-QG14.	E08,J05,A06
K08	Compliance evidence, control testing, and audit readiness	Map controls to MR, CI, registry, access, deployment, monitoring, and incident evidence; schedule control tests and evidence retention.	A sample release can be reconstructed from hypothesis to orders and rollback; evidence gaps become tracked control findings. QG12-QG15.	A07,K01-K07,J07
L01	Cash equities asset certification	Onboard listings, corporate actions, borrow/locates, auctions, halts, tick/lot rules, settlement, market data, costs, risk, venue adapter, and reconciliation.	Golden lifecycle, stress, paper, shadow, and canary cases pass in each target market; market-specific runbooks approved. QG05-QG15.	C-K
L02	Futures asset certification	Onboard contract specifications, multipliers, expiries, rolls, spreads, margin, settlement, limit states, sessions, data, execution, and reconciliation.	Continuous-series research cannot leak roll information; expiry and roll scenarios reconcile to broker/exchange data. QG05-QG15.	C-K
L03	Listed options and volatility certification	Onboard chains, surfaces, greeks, dividends, borrow, exercise/assignment, pin and gap risk, complex orders, margin, and volatility PnL attribution.	Surface and greek validation, expiry/assignment, stress, liquidity, and reconciliation cases pass; risk limits cover nonlinear exposure. QG04-QG15.	C-K
L04	FX, rates, and credit certification	Onboard value dates, calendars, forwards/NDF fixings, curves, day counts, coupons/accruals, calls, settlement, OTC lifecycle, counterparty, margin, and reconciliation.	Convention and lifecycle fixtures pass; curve changes and counterparty limits are versioned; settlement exceptions are controlled. QG04-QG15.	C-K
L05	Digital-assets certification (optional and separately governed)	Onboard 24/7 sessions, exchange/custody risk, forks, token changes, funding, liquidations, stablecoin and counterparty exposure, chain/venue outages, and reconciliations.	Venue credit envelope, custody controls, chain-event runbooks, and continuous operations are independently approved; unsupported venues remain blocked. QG05-QG15.	C-K
L06	Cross-sectional, time-series, and statistical-arbitrage templates	Provide governed templates with target/horizon, universe, features, neutralization, selection-bias controls, turnover, borrow, costs, capacity, portfolio/risk, and monitoring.	At least one nontrivial reference strategy reproduces from ledger to shadow; rejected trials and economics are documented. QG05-QG15.	E-K,L01/L02/L04
L07	Market-making, execution, and volatility-strategy templates	Provide stateful templates for fair value, inventory, quoting, routing, queue, adverse selection, hedging, greeks, latency, venue risk, and kill controls.	Microstructure simulator and live shadow traces calibrate; hard risk remains outside strategy; canary envelope is minimal and explicit. QG04-QG15.	F-K,L01-L04
L08	Institutional readiness review and controlled capital scale-up	Run end-to-end evidence review, DR and game day, operational handover, on-call, independent validation, capital/limit ramp schedule, and post-launch review.	Readiness score meets threshold with no open critical finding; scale increases only after observed stability and signed review; rollback and kill paths remain tested. QG01-QG15.	L01-L07

11.4 Wave exit reviews

At each wave boundary, conduct an evidence review using a real cross-asset sample, not only unit fixtures. The review should include one normal scenario, one data correction, one venue or service failure, one state recovery, one risk breach, one rollback, and one audit reconstruction. Open critical findings block the next capital-exposure milestone.

11.5 Resourcing and sequencing

A practical parallel structure is:

- Platform/build pod: A, B, I foundations.
- Data/reference pod: C, D and asset lifecycle.
- Research/simulation pod: E, F and strategy templates.
- Portfolio/risk/trading pod: G, H and venue certification.
- SRE/security/governance pod: J, K, DR and evidence.
- Asset pods: L, starting with one narrow path once C–K foundations are stable.

Do not create a large separate “productionization team” that rewrites research after hand-off. The shared kernel and joint ownership are designed to remove that translation gap.

Chapter 12

Asset and strategy certification playbooks

12.1 Asset onboarding checklist

An asset class is not onboarded because its prices are in a table. Certification requires identity and lifecycle, calendars and conventions, market/reference data, point-in-time history, accounting, costs, execution protocol, risk, margin/collateral, reconciliation, monitoring, runbooks, and tested failure modes. The asset adapter must expose what is unsupported; silent generic fallbacks are prohibited.

12.2 Strategy manifest

Every strategy package declares forecast target, horizon, universe, feature graph, model, portfolio policy, execution policy, accounts, venues, capital/limit envelope, state schema, data requirements, SLOs, alerts, rollback, and owner. Stateful strategies also declare checkpoint cadence, recovery semantics, and maximum replay lag.

12.3 Strategy-family certification matrix

Family	Research controls	Simulation/risk focus	Live monitoring focus
Cross-sectional alpha	Universe/PIT, neutralization, multiple testing, cross-sectional dependence, turnover/borrow	Capacity, impact, optimizer stability, crowding, sector/factor constraints	Exposure drift, turnover, borrow, TCA, decay by cohort and rebalance.
Time-series/stat arb	Overlapping labels, cointegration/stability, regime splits, multiple horizons, trial ledger	Latency, asynchronous markets, stop/exit, gap risk, financing	Residual distribution, hedge ratio/state, divergence, leg risk, slippage.
Market making	Fair-value decomposition, adverse selection, queue and fill calibration, inventory objective	L2/L3 state, queue, latency, throttles, disconnect, inventory and venue risk	Quote age, fill/reject, queue, adverse selection, inventory, PnL by spread/hedge, clock.
Execution algorithms	Arrival benchmark, counterfactual policy, order-selection bias, venue data	Market impact, timing, participation, routing, child-order state, venue outages	Implementation shortfall, fill, reject, venue toxicity, participation, residual risk.
Volatility/options	Surface construction, dividend/rate/borrow assumptions, nonlinear labels	Greeks, exercise/assignment, pin/gap, liquidity, hedge costs, stress	Greek and surface drift, hedge slippage, volatility PnL, margin, expiry risk.
Macro/event/ML	Event timestamp/knowledge, alternative-data rights, calibration, explanation, regime and trial control	Scenario timing, crowding, sparse events, distribution shift, fallback	Data availability, prediction/calibration drift, regime, concentration, narrative-to-order lineage.

12.4 Capital scaling protocol

Scale is a controlled change, not a business-only parameter. Each increase reviews live TCA versus simulation, market participation, fill quality, impact convexity, borrow/funding, liquidity stress, risk concentration, operational incidents, and limit headroom. A generic capacity curve is

$$\Pi(Q) = \alpha Q - c_s Q - \eta Q^{1+\delta} - c_f Q, \tag{12.1}$$

where impact convexity δ and coefficient η are empirically estimated with uncertainty. Scale increments remain within the validated envelope and are reversible.

Chapter 13

Implementation templates

13.1 Merge request template

```
# Summary
Problem / hypothesis:
Scope and non-goals:
Roadmap ID and linked issue:
Risk class: R0 | R1 | R2 | R3 | R4

# Impact
Affected packages, services, strategies, assets, venues, accounts:
Schema/data/model/config/limit changes:
Security, entitlement, regulatory, and model-risk impact:

# Evidence plan
QG01–QG15 applicability and rationale:
Unit/property/numerical tests:
PIT/leakage and research validation:
Replay/integration/failure scenarios:
Cost/capacity/risk evidence:
Performance and resilience budgets:

# Release plan
Migration and compatibility:
Paper/shadow/canary scope:
Monitoring, alerts, and runbook:
Rollback target and kill procedure:
Post-deploy checks and observation window:

# Approvals
Code Owners:
Independent research/model validation:
Risk / SRE / security / compliance:
Waivers with expiry:
```

13.2 CODEOWNERS example

```
/schemas/                 @domain-architecture @data-platform
/data/                    @data-platform @data-quality
/research/                @quant-research @research-platform
/simulation/              @simulation @quant-research
/portfolio/               @portfolio-research @risk-engineering
/risk/                    @risk-engineering @independent-risk
/trading/oms/             @trading-engineering @independent-risk @sre
/trading/venue_adapters/  @connectivity @trading-engineering @sre
/platform/deploy/         @platform @sre @security
/platform/secrets/        @security @platform
/environments/production/ @sre @security @independent-risk
/assets/options/          @options-domain @risk-engineering
```

13.3 Conceptual CI pipeline

```
stages: [preflight, build, test, quant, integration, performance,
         security, evidence, merged_result, promote]

preflight:
  script: ["tools/ci/preflight --affected"]

hermetic_build:
  script: ["bazel build --config=ci //affected:all"]
```

```
  artifacts: ["provenance.json", "sbom.spdx.json"]

quant_certification:
  rules: ["risk_class >= R2"]
  script:
    - "tools/ci/pit_leakage_suite"
    - "tools/ci/research_protocol_check"
    - "tools/ci/cost_capacity_check"
    - "tools/ci/risk_scenario_check"

release_candidate:
  needs: ["all_required_gates", "required_approvals", "merged_result"]
  script: ["tools/release/create_signed_manifest --commit $CI_COMMIT_SHA"]
```

13.4 Dataset contract example

```
dataset_id: market.equities.canonical_quotes.v3
owner: data-market
schema: schemas/market/quote/v3.proto
source_entitlements: [vendor_a_equities_l1]
time_semantics:
  event_time: exchange_timestamp
  knowledge_time: normalized_receipt_timestamp
  valid_time: null
publication_slo:
  freshness: strategy_tier_specific
quality:
  - no_duplicate(source, instrument_id, sequence)
  - bid_price <= ask_price unless market_state in [locked, crossed]
  - knowledge_time >= ingestion_time_lower_bound
corrections: append_only_revision_stream
retention: policy://market-data-raw-derived
```

13.5 Strategy manifest example

```
strategy_id: equities.cross_sectional.example.v1
owner: alpha-pod-1
kernel_digest: sha256:...
feature_graph: registry://features/example/17
model: registry://models/example/42
forecast:
  target: next_session_residual_return
  horizon: 1d
universe: registry://universes/liquid_equities/8
portfolio_policy: registry://portfolio/neutral_long_short/11
execution_policy: registry://execution/close_auction_plus_vwap/6
risk_envelope: registry://limits/example_canary/3
activation:
  mode: canary
  max_capital: 1000000
  allowed_venues: [venue_x]
monitoring:
  dashboard: observability://strategy/example
  alerts: [data_stale, exposure_breach, tca_degradation, state_divergence]
rollback_release: registry://releases/example/previous
```

13.6 Release manifest example

```
{
  "release_id": "rel-2026-08-25-0017",
  "git_sha": "...",
  "artifact_digest": "sha256:...",
  "build_provenance": "attestation://...",
  "sbom": "spdx://...",
  "model_digest": "sha256:...",
  "feature_graph_digest": "sha256:...",
  "config_digest": "sha256:...",
  "limit_digest": "sha256:...",
  "schema_versions": {"order": 5, "fill": 4, "risk": 7},
  "evidence_bundle": "evidence://mr/12345/current-sha",
  "rollout": {"mode": "shadow", "scope": "..."},
  "rollback_release": "rel-2026-08-10-0009",
  "approvals": ["code_owner", "independent_risk", "sre"]
}
```

Chapter 14

Institutional readiness scorecard

14.1 Maturity levels

Level	Name	Observable condition
0	Ad hoc	Workstation research, manual files, mutable environments, copied production code, weak lineage.
1	Reproducible research	Pinned environment, point-in-time datasets, experiment ledger, deterministic batch output.
2	Controlled simulation	Shared kernel, calibrated costs, accounting/risk invariants, replay and scenario suites.
3	Shadow ready	Immutable artifacts, paper/shadow path, state recovery, independent risk, correlated telemetry.
4	Capital ready	Canary, kill switches, SLOs, DR, independent validation, complete audit evidence.
5	Scalable institutional	Multiple assets/strategies onboard through reusable contracts; measured capacity, low defect escape, regular game days and control testing.

14.2 Final launch checklist

1. All 15 applicable quality gates pass with no critical waiver.
2. Release manifest is signed, immutable, and reports correctly from every live process.
3. Data, feature, model, portfolio, risk, order, and PnL lineage is queryable end-to-end.
4. Replay, paper, shadow, and canary traces have no unexplained material divergence.
5. Independent risk blocks tested breaches and kill switches cancel/disable safely.
6. Crash/restart, venue disconnect, data stale, clock, backpressure, and DR exercises pass.
7. TCA and capacity assumptions are calibrated, conservative, and monitored against live outcomes.
8. Alerts route correctly; runbooks, on-call, escalation, and communications are active.
9. Positions, cash, fills, fees, borrow/funding, and PnL reconcile independently.
10. Model validation, security, compliance, SRE, domain owners, and capital owner sign the bounded activation envelope.

The desired end state

A researcher can specify a hypothesis and a fully versioned experiment; the platform can build a point-in-time dataset, run reproducible validation and simulation, construct a portfolio under risk and cost constraints, package a signed release, promote it through paper/shadow/canary, observe every technical and economic component, stop it independently, recover state, and reconstruct the full evidence chain after the fact.

Appendix A

Failure-mode control matrix

Failure mode	Preventive control	Detection/test	Safe response
Lookahead or revised data	Bitemporal records, as-of API, locked universe, train-only transforms	Adversarial leakage suite, snapshot diff, lineage review	Reject research result; invalidate affected models/releases.
Research/live skew	Shared kernel, generated contracts, identical feature graph	Decision-trace differential test in replay/shadow	Block activation or rollback.
Symbol/lifecycle error	Stable IDs, temporal mappings, mastered reference data	Golden corporate-action/roll fixtures, cross-source reconciliation	Quarantine instrument/asset; reconcile and backfill.
Stale or missing market data	Freshness SLO, redundant sources, explicit data state	Sequence/gap monitor, synthetic probe, clock monitor	Degrade, hedge-only, or stop affected strategy.
Duplicate order after retry	Idempotency key, state fencing, durable OMS	Duplicate/delayed event scenario, drop-copy reconciliation	Block/cancel, recover state, reconcile.
Risk gateway unavailable	Independent replicated service, cached signed limits, fail-safe timeout	Failure injection, health and decision-latency SLO	Fail closed for new risk; cancel/hedge according to policy.
Config drift	Typed config, immutable digest, GitOps/host attestation	Runtime self-report versus registry, drift monitor	Reject activation or roll back.
Cost/capacity underestimation	Calibrated segmented models, conservative stress	Simulated vs realized TCA and participation curves	Reduce capital/urgency; recalibrate before scale.
Model drift or regime change	Feature/model monitoring and regime indicators	Drift, calibration, residual, exposure and PnL attribution	Review, reduce/disable, retrain only through full promotion.
Venue/session failure	Certified adapter, recovery state machine, alternate routing where approved	Disconnect/sequence/reset game day, synthetic orders	Cancel, reroute, venue kill, reconcile.
Clock degradation	PTP/NTP health, monotonic clocks, timestamp quality	Offset/jitter alerts, cross-system event-order checks	Block latency-sensitive activation; degrade/stop.
Artifact compromise	Hermetic build, provenance, SBOM, signature, admission policy	Tamper test, signature/provenance verification	Reject artifact, revoke keys, rotate and investigate.
Partial deployment/migration	Progressive rollout, compatibility, expand-migrate-contract	Mixed-version integration and rollback scenario	Halt rollout; roll back compatible set.
Position/PnL divergence	Independent position/cash and drop-copy reconciliation	Intraday breaks, ledger conservation, broker/clearing compare	Stop new risk, reconcile, correct under dual control.
Operator error	Least privilege, typed controls, previews, dual authorization	Audit stream, control-plane validation, game day	Abort/rollback, preserve evidence, review access and UX.

Appendix B

Selected references

Bibliography

[1] Jane Street. “How Jane Street Does Code Review.” Public technical talk. https://www.janestreet.com/tech-talks/janestreet-code-review/

[2] Jane Street. “Building reproducible Python environments with XARs.” https://blog.janestreet.com/building-reproducible-python-environments-with-xars/

[3] Hudson River Trading. “Work at HRT.” Public description of algorithm development, core engineering, research, live trading, and risk infrastructure. https://www.hudsonrivertrading.com/work-at-hrt/

[4] Citadel Securities. “Engineering.” Public description of distributed trading systems, simulation, research/data platforms, risk systems, and SRE. https://www.citadelsecurities.com/careers/engineering/

[5] Potvin, R. and Levenberg, J. “Why Google Stores Billions of Lines of Code in a Single Repository.” Communications of the ACM, 2016. https://research.google/pubs/why-google-stores-billions-of-lines-of-code-in-a-single-repository/

[6] Bazel Documentation. “Hermeticity.” https://bazel.build/basics/hermeticity

[7] NixOS Foundation. “Nix and NixOS: Declarative builds and deployments.” https://nixos.org/

[8] GitLab Documentation. “Merge request approvals.” https://docs.gitlab.com/user/project/merge_requests/approvals/

[9] GitLab Documentation. “Code Owners.” https://docs.gitlab.com/user/project/codeowners/

[10] GitLab Documentation. “Merged results pipelines.” https://docs.gitlab.com/ci/pipelines/merged_results_pipelines/

[11] GitLab Documentation. “Merge trains.” https://docs.gitlab.com/ci/pipelines/merge_trains/

[12] Bailey, D. H., Borwein, J. M., Lopez de Prado, M., and Zhu, Q. J. “The Probability of Backtest Overfitting.” Journal of Computational Finance, 2017; working-paper version available at SSRN. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2326253

[13] Bailey, D. H. and Lopez de Prado, M. “The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting and Non-Normality.” Journal of Portfolio Management, 2014. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2460551

[14] Harvey, C. R., Liu, Y., and Zhu, H. “... and the Cross-Section of Expected Returns.” Review of Financial Studies, 29(1), 2016. https://academic.oup.com/rfs/article/29/1/5/1843824

[15] Novy-Marx, R. and Velikov, M. “A Taxonomy of Anomalies and Their Trading Costs.” Review of Financial Studies, 29(1), 2016. https://academic.oup.com/rfs/article/29/1/104/1844518

[16] Board of Governors of the Federal Reserve System, FDIC, and OCC. “Supervisory Guidance on Model Risk Management,” SR 26-2, 17 April 2026. This supersedes SR 11-7 for the stated agencies. https://www.federalreserve.gov/supervisionreg/srletters/SR2602.htm

[17] Financial Conduct Authority. “Algorithmic Trading Controls: Multi-Firm Review.” 2025. https://www.fca.org.uk/publications/multi-firm-reviews/algorithmic-trading-controls

[18] U.S. Securities and Exchange Commission. “Risk Management Controls for Brokers or Dealers with Market Access,” Rule 15c3-5. https://www.sec.gov/files/rules/final/2010/34-63241.pdf

[19] FIX Trading Community. “Automated Trading and Risk Management.” https://fixtrading.org/guidelines/automated-trading/

[20] National Institute of Standards and Technology. “Secure Software Development Framework (SSDF) Version 1.1,” SP 800-218, 2022. https://csrc.nist.gov/pubs/sp/800/218/final

[21] OpenSSF. “SLSA Specification v1.2.” https://slsa.dev/spec/v1.2/

[22] SPDX Project. “System Package Data Exchange.” ISO/IEC 5962:2021. https://spdx.dev/

[23] Sigstore. “Cosign: signing and verifying artifacts and attestations.” https://docs.sigstore.dev/cosign/

[24] OpenTelemetry. “Documentation.” https://opentelemetry.io/docs/

[25] Prometheus. “Instrumentation and metric best practices.” https://prometheus.io/docs/practices/instrumentation/

[26] Google. “Site Reliability Engineering: Service Level Objectives.” https://sre.google/sre-book/service-level-objectives/

[27] Argo Project. “Argo Rollouts: Progressive Delivery for Kubernetes.” https://argo-rollouts.readthedocs.io/

[28] Apache Iceberg. “Evolution and time travel documentation.” https://iceberg.apache.org/docs/latest/

[29] MLflow. “Model Registry.” https://mlflow.org/docs/latest/ml/model-registry/

[30] OpenLineage. “Open standard for data lineage.” https://openlineage.io/docs/